

# Fetch API & JSON

Connect Your App to Any Backend

## ■ FETCH & APIS

### ■ LEARNING OUTCOME

By the end of this module you will make GET/POST/PUT/DELETE requests, handle JSON, add auth headers, build a reusable API client, work with FormData for file uploads.

#### ■ Skills You Will Gain

- Make all HTTP methods with Fetch
- Parse and send JSON correctly
- Add authentication headers
- Handle network vs HTTP errors
- Build a reusable API client
- Work with FormData for uploads

■ Every modern web app fetches data from APIs. Fetch + JSON is the universal language of the web. Master this and you can connect to any backend on Earth.

## SECTION 1

# Fetch Fundamentals and CRUD

```
// FETCH returns Promise<Response> -- only rejects on NETWORK failure! // HTTP errors (404, 500)
resolve normally -- check response.ok! async function safeFetch(url, options={}) { const res =
await fetch(url, options); if (!res.ok) throw new Error(`HTTP ${res.status}: ${res.statusText}`);
return res.json(); } // GET const users = await safeFetch('/api/users?page=1&limit=20'); // POST --
create new resource async function createUser(data) { return safeFetch('/api/users', { method:
'POST', headers: { 'Content-Type':'application/json', 'Authorization':`Bearer ${token}` }, body:
JSON.stringify(data) }); } // PUT -- full replacement async function updateUser(id, data) { return
safeFetch(`/api/users/${id}`, { method: 'PUT', headers: { 'Content-Type':'application/json' },
body: JSON.stringify(data) }); } // PATCH -- partial update async function patchUser(id, changes) {
return safeFetch(`/api/users/${id}`, { method: 'PATCH', headers: {
'Content-Type':'application/json' }, body: JSON.stringify(changes) // only fields to change }); }
// DELETE async function deleteUser(id) { const res = await fetch(`/api/users/${id}`, { method:
'DELETE', headers: { 'Authorization':`Bearer ${token}` } }); if (!res.ok) throw new Error(`Delete
failed: ${res.status}`); return res.status === 204 ? null : res.json(); // 204 has no body }
```

## SECTION 2

# Reusable API Client and JSON

```
// Reusable API client class class APIClient { #baseUrl; #headers; constructor(baseUrl) {
this.#baseUrl = baseUrl; this.#headers = { 'Content-Type':'application/json',
'Accept':'application/json' }; } setToken(t) { if(t) this.#headers['Authorization']=`Bearer ${t}`;
else delete this.#headers['Authorization']; } async #req(path, opts={}) { const res = await
fetch(this.#baseUrl + path, { ...opts, headers:{...this.#headers,...opts.headers} }); if
(res.status===401) { this.setToken(null); throw new Error('Session expired'); } const body = await
res.json().catch(()=>null); if (!res.ok) throw Object.assign(new Error(body?.message||`HTTP
${res.status}`), {status:res.status}); return body; } get(path, params) { const q=params?''+new
URLSearchParams(params):''; return this.#req(path+q); } post(path, data) { return
this.#req(path,{method:'POST',body:JSON.stringify(data)}); } put(path, data) { return
this.#req(path,{method:'PUT', body:JSON.stringify(data)}); } patch(path, data) { return
this.#req(path,{method:'PATCH',body:JSON.stringify(data)}); } delete(path) { return
this.#req(path,{method:'DELETE'}); } upload(path, file, extra={}) { const form = new FormData();
form.append('file', file); Object.entries(extra).forEach(([k,v])=>form.append(k,v)); const
{['Content-Type':_,...h} = this.#headers; // remove CT -- browser sets multipart return
this.#req(path,{method:'POST',headers:h,body:form}); } } const api = new
APIClient('https://api.example.com/v1'); api.setToken(localStorage.getItem('token')); const users =
await api.get('/users',{page:1,limit:20}); // JSON const json = JSON.stringify(obj); // to string
const pretty = JSON.stringify(obj, null, 2); // pretty 2-space const parsed = JSON.parse(json); //
back to object const safe = JSON.parse(str) ?? {}; // never throws // JSON does NOT support:
undefined, Function, Date (-> ISO string), Map, Set // Safe parse helper function safeJSON(str,
fallback=null) { try { return JSON.parse(str); } catch { return fallback; } }
```

## ■ PRACTICE Q & A

Q1. Why does fetch() not throw for 404 or 500?

**A:** fetch() only rejects on NETWORK failures (no internet, CORS blocked). HTTP error status codes are successful responses to fetch -- you must check response.ok or response.status manually.

Q2. What is the difference between PUT and PATCH?

**A:** PUT: FULL replacement -- send entire updated object. Server replaces resource completely. PATCH: PARTIAL update -- send only fields to change. Use PATCH for efficiency.

Q3. What happens when you JSON.stringify a Date object?

*A: It converts to ISO 8601 string: '2024-01-15T10:30:00.000Z'. When parsing back, it comes as a string NOT a Date object. Use a reviver function to convert back.*

**Q4.** How do you send a file upload with fetch?

*A: Use FormData: `const form = new FormData(); form.append('file', input.files[0]); fetch(url, {method:'POST', body:form})`. Do NOT set Content-Type manually -- browser sets multipart/form-data with boundary automatically.*

**Q5.** What is CORS and why does it matter?

*A: Cross-Origin Resource Sharing: browsers block requests to a DIFFERENT origin unless the server allows it with CORS headers. Fix: server adds Access-Control-Allow-Origin header, or use a same-origin proxy.*

| ■ Fetch API Cheat Sheet                         |                                          |
|-------------------------------------------------|------------------------------------------|
| <code>fetch(url)</code>                         | GET request -- returns Promise<Response> |
| <code>response.ok</code>                        | True if status 200-299                   |
| <code>response.status</code>                    | HTTP status code                         |
| <code>await response.json()</code>              | Parse JSON body                          |
| <code>await response.text()</code>              | Get body as string                       |
| <code>{ method: 'POST' }</code>                 | Set HTTP method                          |
| <code>body: JSON.stringify(data)</code>         | Send JSON body                           |
| <code>'Content-Type': 'application/json'</code> | JSON content type header                 |
| <code>'Authorization': 'Bearer token'</code>    | Auth header                              |
| <code>new FormData()</code>                     | For file uploads                         |
| <code>JSON.stringify(obj, null, 2)</code>       | Pretty-print JSON                        |
| <code>JSON.parse(str)</code>                    | JSON string to object                    |